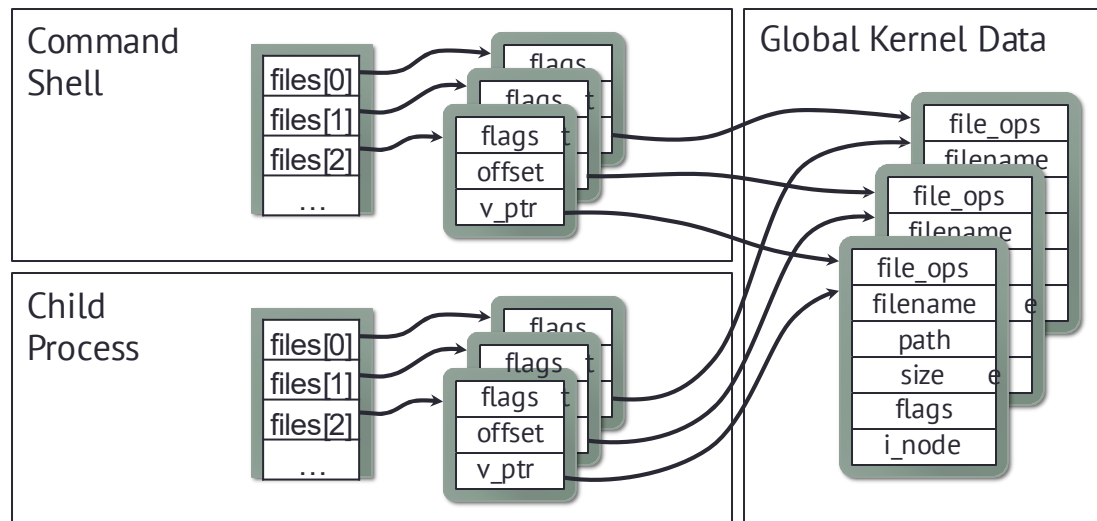


Command Shell and Child Process

- How does the child process output to the command shell's standard output?
- How does it get the shell's stdin?
- When a UNIX process is forked, it is a near-identical copy of the parent process
 - Only differences: process ID and parent process ID
- Specifically, the child process has the same files open as the parent process
 - And they have the exact same file descriptors

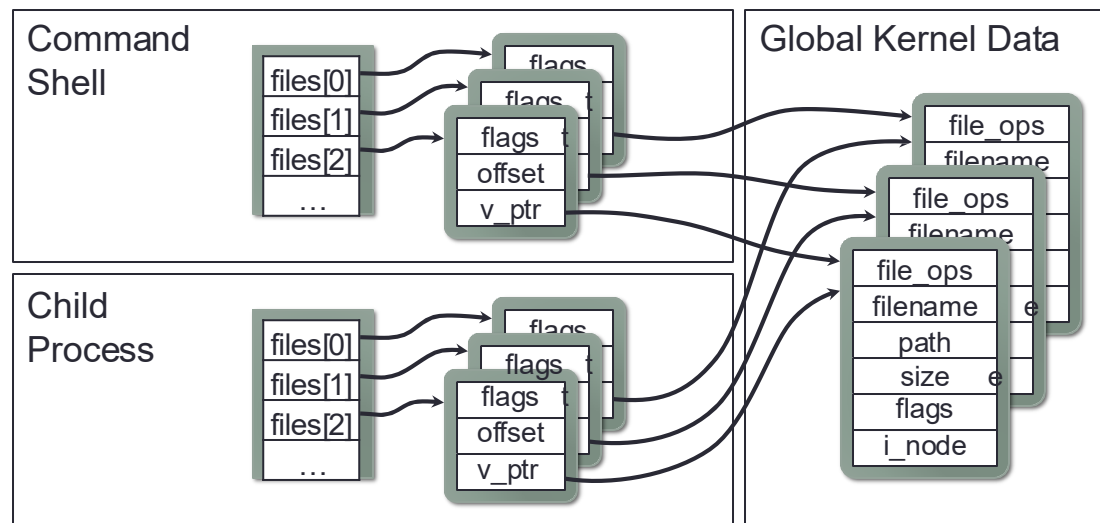
Command Shell and Child Process

When child process reads stdin and writes stdout/stderr, it writes the exact same files that the command-shell has as **stdin/stdout/stderr**



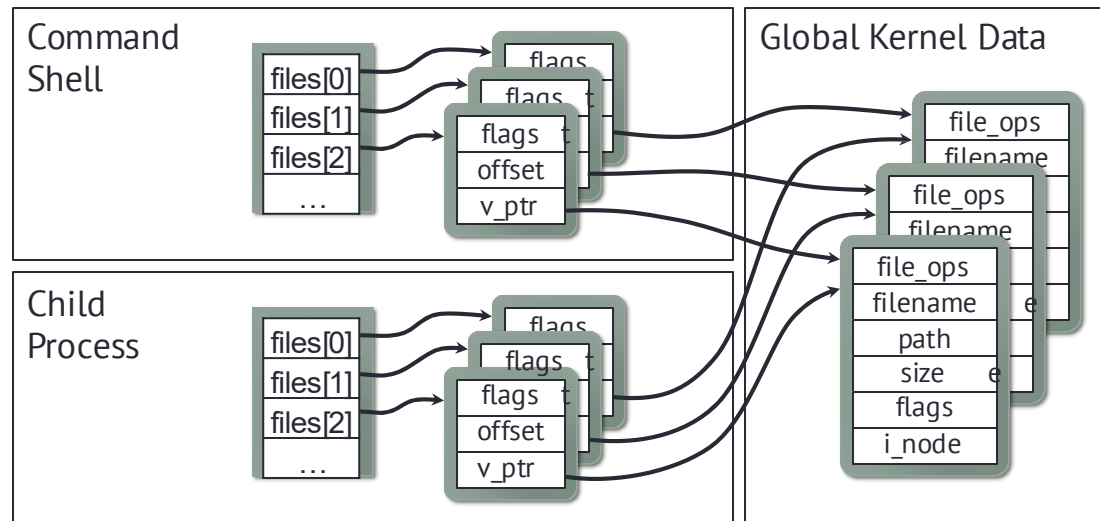
Command Shell and Child Process

- If command redirects e.g., the output to a file, clearly can't have the command-shell process do it before forking
 - Would work fine for the child process, but the command-shell's I/O state would be broken for subsequent commands



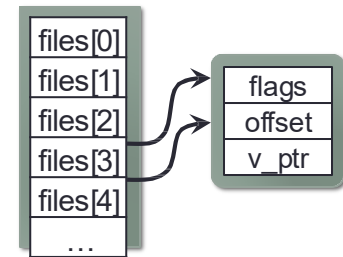
Command Shell and Child Process

- Child process must set up **stdin**, **stdout**, and **stderr** before it executes the actual program
- How does a process change what file is referenced by a given file descriptor?
 - Process must ask the kernel to modify the file descriptors



Manipulating File Descriptors

- UNIX provides two system calls: **dup()** and **dup2()**
- **int dup(int filedes)**
 - Duplicates the specified file descriptor, returning a new, previously unused file descriptor
- Note that the internal **file** struct is **not** duplicated, only the pointer to the file struct!
- **Implication:**
 - **Reads, writes and seeks through both file descriptors affect a single shared file-offset value**
- Even though the one file has two descriptors, should call **close()** on each descriptor
 - Remember: each process has a maximum number of open files
 - (Kernel won't free the file struct until it has no more references)



Manipulating File Descriptors

- **int dup2(int fildes, int fildes2)**
 - Duplicates the specified file descriptor into the descriptor specified by **fildes2**
 - If fildes2 is already an open file, it is closed before **dup2()** duplicates **fildes**
 - (Unless **fildes == fildes2**, in which case nothing is closed)
- This function allows the command-shell's child process to redirect standard input and output
 - e.g. to replace **stdout** with a file whose descriptor is in **fd**:
dup2(fd, STDOUT_FILENO);
- As before, the file descriptor that was duplicated should be closed to keep from leaking descriptors
close(fd);

Manipulating File Descriptors

- Previous example:

```
grep Allow < logfile.txt > output.txt
```

- After command shell forks off a child process, the child can execute code like this, before it starts grep:

```
int in_fd, out_fd;
```

```
in_fd = open("logfile.txt", O_RDONLY);  
/* Replace stdin */  
dup2(in_fd, STDIN_FILENO);  
close(in_fd);
```

```
out_fd = open("output.txt", O_CREAT | O_TRUNC | O_WRONLY, 0);  
/* Replace stdout */  
dup2(out_fd, STDOUT_FILENO);  
close(out_fd);
```

The Sandbox...

- You have a lot of bad code that take input from **hostile users**
 - Often written in C/C++
- The sandbox generally covers an entire process
 - That way one can take advantage of operating-specific features that allow a process to restrict what it is allowed to do

Why Sandbox at all?

- The sandbox is mostly good at making C/C++ memory exploits no longer exploitable
 - Now the attacker needs to both exploit the C code AND exploit a weakness in the sandbox
- Defence in depth...
 - But why bother with defence in depth?
- Because it is cheaper!
 - Cheaper to keep using the same bad C and C++ code and put it in a letterbox than it is to actually rewrite the code in a secure language!

The Broker and the Target

- Most sandboxes separate out the problem into separate components
 - That run as separate processes
- The **Broker** is the reference monitor/trusted computing base
 - Its job is to start up the targets
 - **ALL** requests for anything sensitive in the target must be passed to the broker
- The target is the sandboxed code
 - Establish communication with the broker
 - Provide an API for talking to the broker
 - And then yield all other privileges

Don't Reinvent the Wheel: Just Download Someone Else's!

- For Windows:
 - Wrapper around Chrome's sandbox
 - Mozilla Sandbox
 - (<https://wiki.mozilla.org/Security/Sandbox>)
- For Linux:
 - Uses seccomp as the building block

Old School Unix Sandboxing: the chroot jail

- People have wanted sandboxes for a long time...
 - Far longer than the OSs have provided fine grained mediation necessary to create sandboxes
- The gen-1 Unix Sandbox:
 - The **chroot** system call changes the definition / for the invoking process
- This enforces the following property:
 - A process (and all processes it invokes, directly or indirectly) can not read or write to any new file outside the new **root** directory
 - But can still access existing files

Limitations of chroot

- It is a privileged operation! Because you can do things that would compromise the system otherwise:
 - Create a directory with a file name etc/sudoers with the appropriate context
 - Now chroot to that directory
 - Now invoke sudo
 - Voila, you have root!
 - So, any program using chroot must then drop its privileges to run as "nobody" or an otherwise unknown user
- It does not affect **system call** operation
 - So a "jailed" process can still access the network, call the kernel (and therefore perhaps kernel bugs).
 - The "nobody" account actually still has privileges! Like the ability to interfere with other processes also owned by the "nobody" user

“Modern” Linux sandboxing: seccomp

- Desired property:
- An application can read and write to specified "file descriptors", but otherwise invoke no other system calls
 - As we've seen, file descriptors are more than just files: they are general communication pipes between processes
- **seccomp** enables a process to lock itself out of the system call interface:
 - Enables **read()** and **write()** on open file descriptors
 - Enables **exit()**
 - Everything else is blocked
 - An alternative form allows specifying a filter to only block some system calls

Using seccomp to create a sandbox

- Split things conceptually into a **broker** and one or more **targets**
 - The targets are the sandboxed elements
 - The broker controls the targets and is the trusted base
- The broker starts up the targets
 - Using fork or clone
 - Establishes communication channels to the targets
 - Starts the target processes running
- The target process then invokes **seccomp** to give up any privileges

Process communication

- Standard Unix pipes/file descriptors
 - Shove bits to/from the sandboxed process
 - Just like a network program
- Shared Memory
 - A common pool of memory that both can read and write to
 - Need to also use a semaphore to ensure coherency

Software using seccomp (or seccomp-bpf)

- **Android:** uses a seccomp-bpf filter in the zygote since Android 8.0 Oreo.
- **Systemd:** sandboxing options are based on seccomp.
- **QEMU:** the Quick Emulator, the core component to the modern virtualization together with KVM uses seccomp
- **Docker:** Docker can associate a seccomp profile with the container using the `--security-opt` parameter.
- **Chrome:** seccomp-bpf is used to sandbox the renderers.
- **OpenSSH:** has supported seccomp-bpf since version 6.0.
- **Mbox:** a lightweight sandboxing mechanism that uses *ptrace* along with seccomp-bpf to create a secure sandbox with less overhead than **ptrace** alone.
- **LXD:** a Ubuntu "hypervisor" for containers
- **Firefox and Firefox OS**
- **Tor:** since 0.2.5.1-alpha

Use case for VMs

- **Isolation & Sandboxing**
 - **Running untrusted (or dangerous) code**
 - **Allowing untrusted users access to an isolated environment**
- **Resource Allocation**
 - Limit the maximum amount of resources an application or user may use
 - You might rent only part of the compute power available to you
- **System Management**
 - It is very easy to back up and restore an entire VM; shut it down, restart it, etc, without losing access to the host computer.
- **Cross-platform testing**
 - Test your apps on other operating systems without using an entire computer, or needing to reinstall a computer you're already using.
- **Prototyping**
 - Virtualize an entire network with multiple VMs and emulated routers/switches/etc. to prototype an entire network!
- **Emulation**
 - Emulate entirely different CPU architectures - slower - but allows for an incredibly diverse range of applications (including legacy applications) to run on your computer.

Emulation vs Virtualization

- **Emulation** involves recreating and modelling fully different computer architectures – basically mimicking hardware.
 - This is necessary when the operating system or software you're trying to run are written for a different class of hardware than the host machine.
- Examples:
 - **32bit** architectures running in **64bit**: different hardware
 - **arm** running on **x86**: different instruction sets
 - **Linux** binaries running on **Windows** (or the other way around): different binary formats.

Emulation is Not Security

- Emulation primitives (e.g. Windows Subsystem for Linux, wine) and virtual machines are not primarily designed as security sandboxes!
 - Relying on something not designed for sandboxing can be a problem!
 - N.B. security in QEMU is provided by seccomp

Emulation vs Virtualization

- **Emulation** involves recreating and modelling different computer architectures – basically mimicking hardware.
 - This is necessary when the operating system or software you're trying to run are written for a different class of hardware (or OS) than the host machine.
- **Virtualization** works when the OS (or software) is built for the same class of hardware as the host machine.
 - Virtualization means “making one thing look or act like multiple things.”
 - In this case, it means your computer hardware!
 - Virtualization makes use of your computer hardware directly and is therefore faster than emulation. Usually by a lot.

Virtualization is supported by hardware

In order to provide virtual machines with complete CPU access and ensure security of the host system, CPU manufacturers have extensions (special instructions!):

- Intel VT-x
- AMD-V

The role of hypervisors

- Okay, so CPUs have extensions for virtualization; do hypervisors need to do anything else interesting?
 - Yes!
- There's more to provide to the guest than just CPU & Memory:
 - network access? → Virtual Networks
 - hard drive space? → Virtual Hard Drives
 - CD-ROM access (yes still!) → Disk Images

Bare Metal vs Hosted Hypervisors

Two types of hypervisors: **bare metal** (type 1) and **hosted** (type 2).

- Bare Metal hypervisors is when the hypervisor is the host.
 - This means the hypervisor is the “operating system.”
- Hosted hypervisors are programs that run on a host operating system.
 - We’re using type 2 hypervisors when we use UTM and VirtualBox

Let’s try to put these in the context of OS operation and security.

Hypervisors

- **Bare Metal Hypervisors**

- VMWare ESXi
- Hyper-V
- Proxmox
- Xen

- **Hosted Hypervisors**

- VirtualBox
- VMWare Workstation
- UTM (uses qemu under the hood)
- qemu (also supports emulation)
- Hyper-V
- Parallels